

Sistemes Distribuïts en Xarxa (SDX)
Facultat d'Informàtica de Barcelona
Control parcial. 6 d'Abril 2022

Contesteu a les preguntes de manera concisa i precisa
Contesteu al mateix full
Test, seminaris: no es poden consultar apunts
Readings: podeu portar els vostres informes
Problemes: podeu portar un full de notes imprès a doble cara
Durada: 1 hora, 50 minuts

Nom i Cognoms:

1. (5 pts) Seleccioneu la resposta (una només) que considereu correcta en cadascun dels apartats. Cada resposta correcta val 1/2 punts. Cada resposta incorrecta resta 1/6.
 - (a) Quina de les següents aplicacions no necessita que el middleware li exposi la ubicació dels diferents usuaris que composen el sistema distribuït?
 - ☐ L'aplicació Radar COVID per determinar els usuaris que han estat en contacte estret amb un positiu
 - ☐ L'aplicació Google Maps per recomanar itineraris cap a les destinacions seleccionades als usuaris a partir de les seves ubicacions
 - ☐ L'aplicació Google Calendar per enviar invitacions a reunions a usuaris situats en diferents zones horàries
 - ✓ ☒ L'aplicació Google Docs per permetre l'edició col·laborativa d'un document entre usuaris situats en diferents ubicacions
 - (b) El sistema de control de vol d'un avió inclou múltiples sensors per mesurar l'altitud. Quin dels següents tipus de fallada es produiria si dos sensors reportessin en un mateix moment valors d'altitud diferents?
 - ☐ Timing failure
 - ✓ ☒ Arbitrary (Byzantine) failure
 - ☐ Omission failure
 - ☐ State transition failure
 - (c) Quin dels següents paradigmes de comunicació permet transferir un procés en execució d'un node a un altre per què continuï operant en el node destí?
 - ✓ ☒ Mobile agents
 - ☐ Mobile code
 - ☐ Shared-memory
 - ☐ Remote invocation
 - (d) Quina de les següents tècniques per fer front a fallades durant l'execució de RPCs no necessita que els servidors tinguin estat (i.e., siguin *stateful*)?
 - ☐ Retransmission of results
 - ☐ Duplicate filtering
 - ✓ ☒ Retry request message
 - ☐ Recoverable processes

- (e) Tenim un sistema de publicació/subscripció amb 10 nodes $\{n_0, \dots, n_9\}$ que usa un protocol de *routing* d'events basat en *rendezvous*, on el node n_3 està subscrit als events ($X < 1$) i el node n_8 publica l'event ($X = 0$). Si la funció $SN(X < 1)$ retorna els nodes (n_5, n_7, n_9) i la funció $EN(X = 0)$ retorna els nodes (n_1, n_4, n_5) , quin node enviarà la notificació de l'event ($X = 0$) al node n_3 ?
- ☐ El node que publica l'event, és a dir, n_8
 - ☐ Pot ser qualsevol dels nodes retornats per la funció $SN(X < 1)$, és a dir, (n_5, n_7, n_9)
 - ☐ Pot ser qualsevol dels nodes retornats per la funció $EN(X = 0)$, és a dir, (n_1, n_4, n_5)
 - ✓ El node resultant de la intersecció entre $SN(X < 1)$ i $EN(X = 0)$, és a dir, n_5
- (f) Un client vol sincronitzar el seu rellotge mitjançant l'algorisme de *Cristian*. Envia dos peticions simultànies a dos servidors diferents A i B . La resposta de A inclou un temps del servidor de 06:23:25.500 i el seu temps de *round-trip* ha estat 60 ms. La resposta de B inclou un temps del servidor de 06:23:25.300 i el seu temps de *round-trip* ha estat 50 ms. Si la latència mínima entre el client i els servidors A i B és de 25 i 15 ms, respectivament, i) amb quin servidor hauria de sincronitzar el client el seu rellotge per tal de fer servir el temps més precís, ii) a quin valor hauria de fixar el client el seu rellotge?
- ✓ i) A ; ii) 06:23:25.530
 - ☐ i) A ; ii) 06:23:25.500
 - ☐ i) B ; ii) 06:23:25.325
 - ☐ i) B ; ii) 06:23:25.350
- (g) En l'algorisme de *Lin* d'exclusió mútua, què ha de fer un coordinador quan rep un missatge **Release** d'un procés diferent del que ell havia votat?
- ☐ Aquesta situació no pot passar, ja que el missatge **Release** s'envia només als coordinadors que havien votat el procés
 - ☐ El coordinador pot ignorar el missatge **Release** donat que no havia votat al procés
 - ✓ El coordinador ha de treure el procés de la cua de processos que esperen que els voti
 - ☐ El coordinador ha d'eliminar el seu vot i votar pel primer procés de la cua de processos
- (h) En quin cas les eleccions mitjançant l'algorisme de *Chang & Roberts* i l'algorisme en anell millorat (*Enhanced Ring algorithm*) requereixen el mateix nombre de missatges per finalitzar? (pots assumir que no hi ha fallades i hi ha una única elecció en curs)
- ☐ Aquests algorismes sempre requereixen el mateix nombre de missatges
 - ✓ Quan el procés que inicia l'elecció és el procés amb l'identificador més gran
 - ☐ Quan el procés que inicia l'elecció és el predecessor del procés amb l'identificador més gran
 - ☐ Mai, l'algorisme de *Chang & Roberts* sempre requereix més missatges
- (i) Un procés que s'està comunicant mitjançant multicast atòmic basat en vistes (*view-synchronous atomic multicast*) es troba actualment a la vista G_i . Quin serà el comportament d'aquest procés si rep (en aquest ordre) un missatge per canviar a la vista G_{i+1} i un missatge ordinari m que pertany a la vista G_i ?
- ✓ Posposarà el canvi de vista G_{i+1} per fer-lo després de lliurar el missatge m
 - ☐ Descartarà el canvi de vista G_{i+1} i lliurarà el missatge m
 - ☐ Canviarà a la vista G_{i+1} i descartarà el missatge m
 - ☐ Canviarà a la vista G_{i+1} i tot seguit lliurarà el missatge m
- (j) Quin dels següents és un model de consistència centrat en el client (*client-centric*)?
- ☐ Sequential consistency
 - ☐ Linearizability
 - ☐ Strict consistency
 - ✓ Monotonic Writes

Nom i Cognoms:

2. (SEMINARIS) Contesta les següents preguntes referents als seminaris **Muty** i **Chatty**.

- a) (7 pts) **Muty**. Modifica el següent extracte de codi corresponent a la implementació de l'exclusió mútua de Ricart & Agrawala amb prioritats (mòdul *lock2*), per tal que es comporti conforme a la implementació que utilitza rellotges lògics de Lamport (mòdul *lock3*).

```
wait(Nodes, Master, Refs, Waiting, MyId) ->
receive
  {request, From, Ref, ReqId} ->
  if
    ReqId < MyId ->
      From ! {ok, Ref},
      R = make_ref(),
      From ! {request, self(), R, MyId},
      wait(Nodes, Master, [R|Refs], Waiting, MyId);
    true ->
      wait(Nodes, Master, Refs, [{From, Ref}|Waiting], MyId)
  end;
  {ok, Ref} ->
    NewRefs = lists:delete(Ref, Refs),
    wait(Nodes, Master, NewRefs, Waiting, MyId)
end.

% MyClock: Logical clock of this process
% MyReqTime: Timestamp of the request of this process
wait(Nodes, Master, Refs, Waiting, MyId, MyReqTime, MyClock) ->
receive
  {request, From, Ref, ReqId, ReqTime} ->
    MyNewClock = max(MyClock, ReqTime),
    if
      (ReqTime < MyReqTime) or ((ReqTime == MyReqTime) and (ReqId < MyId)) ->
        From ! {ok, Ref},
        wait(Nodes, Master, Refs, Waiting, MyId, MyReqTime, MyNewClock);
      true ->
        wait(Nodes, Master, Refs, [{From, Ref}|Waiting], MyId, MyReqTime, MyNewClock)
    end;
  {ok, Ref} ->
    NewRefs = lists:delete(Ref, Refs),
    wait(Nodes, Master, NewRefs, Waiting, MyId, MyReqTime, MyClock)
end.
```

- b) (3 pts) **Chatty**. Compara l'escalabilitat, la tolerància a fallades i la latència de comunicació de la implementació en què tots els clients estan connectats a un únic servidor (mòdul *server*) respecte aquella en què els clients estan connectats a servidors diferents (mòdul *server2*).

Solution:

Scalability: A single server limits the amount of clients that can be connected according to its capacity, therefore, *server* implementation scales worse.

Fault tolerance: When there are several servers, only the clients connected to a crashed server will be affected, the others will be able to communicate with each other normally, therefore, *server2* implementation is more fault tolerant.

Latency: When clients are connected to different servers, messages are routed through one additional node (we have $C_i - S_x - S_y - C_j$ instead of $C_i - S - C_j$), therefore, *server2* implementation has more latency.

3. (SEMINARIS) Contesta les següents preguntes referents al seminari **Ordy**.

- a) (7 pts) Completa el següent extracte de codi corresponent a la implementació de multicast causalment ordenat. Fixa't que també cal mantenir actualitzat el rellotge vectorial del procés afegint crides a la funció `incrementVC/2` on correspongui.

```
% checkMsg(Id, MsgVC, VC, N): Check message from process Id with clock MsgVC
% deliverReadyMsgs(Master, VC, Queue, Queue): Deliver ready messages in Queue to Master
server(Id, Master, Nodes, VC, Queue) ->
receive
{send, Msg} ->
    NewVC = incrementVC(Id, VC),
    lists:foreach(fun(Node) ->
        Node ! {multicast, Msg, Id, NewVC} end, Nodes),
    Master ! {deliver, Msg},
    server(Id, Master, Nodes, NewVC, Queue);
{multicast, Msg, FromId, MsgVC} ->
    case checkMsg(FromId, MsgVC, VC, size(VC)) of
        ready ->
            Master ! {deliver, Msg},
            NewVC = incrementVC(FromId, VC),
            {NewerVC, NewQueue} = deliverReadyMsgs(Master, NewVC, Queue, Queue),
            server(Id, Master, Nodes, NewerVC, NewQueue);
        wait ->
            server(Id, Master, Nodes, VC, [{FromId, MsgVC, Msg}|Queue])
    end
end.

% incrementVC(N, VC): Increment position N of vector clock VC
incrementVC(N, VC) ->
    setelement(N, VC, element(N, VC)+1).
```

- b) (1.5 pts) Justifica si els posts es mostren en ordre FIFO i en ordre causal si el sistema multicast està implementat amb el mòdul *total*.

Solution: This version does not necessarily provide FIFO nor causal order, since any two messages sent by the same process are delivered in an arbitrary total order influenced by communication delays.

- c) (1.5 pts) Si modifiquem la implementació del mòdul *total* per tal que proporcioni també una ordenació FIFO, justifica si proporcionaria també una ordenació causal com a conseqüència.

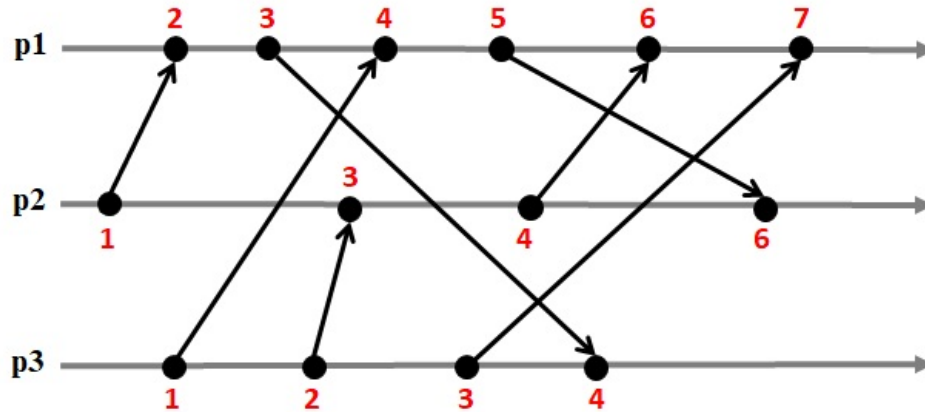
Solution: Yes, it would provide causal ordering. We can prove it by induction from the two basic cases of the happened-before relation:

1. Process *p* sends *m1* and then *m2*. As the multicast is FIFO-ordered, all the processes will deliver *m1* before *m2*.
2. Process *p* sends *m1*, process *q* delivers *m1* and then sends *m2*. Process *q* will propose for *m2* a sequence number which is greater than the agreed number for *m1*, therefore the agreed number for *m2* will certainly be also greater, and all the processes will deliver *m1* before *m2*.

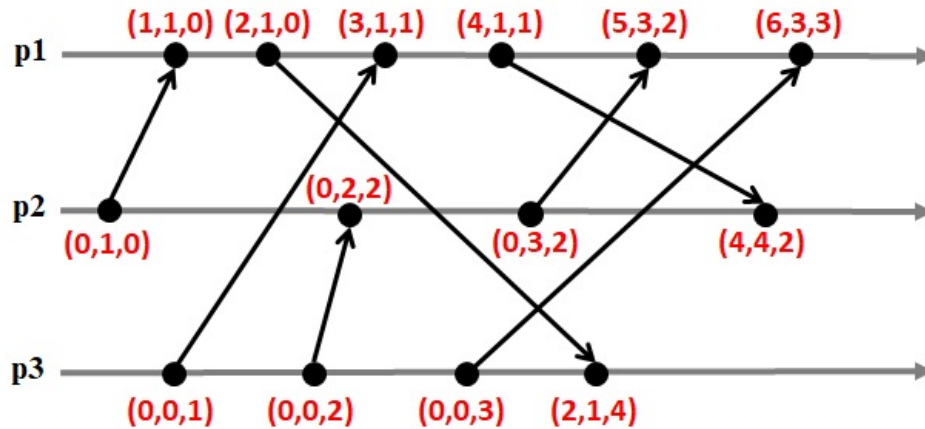
Nom i Cognoms:

4. (1.25 pts) Donada la següent seqüència d'events executada pels processos p_1 , p_2 i p_3 .

a) Etiqueta cada event amb el valor del seu rellotge lògic escalar (i.e. rellotge de Lamport).



b) Etiqueta cada event amb el valor del seu rellotge lògic vectorial.

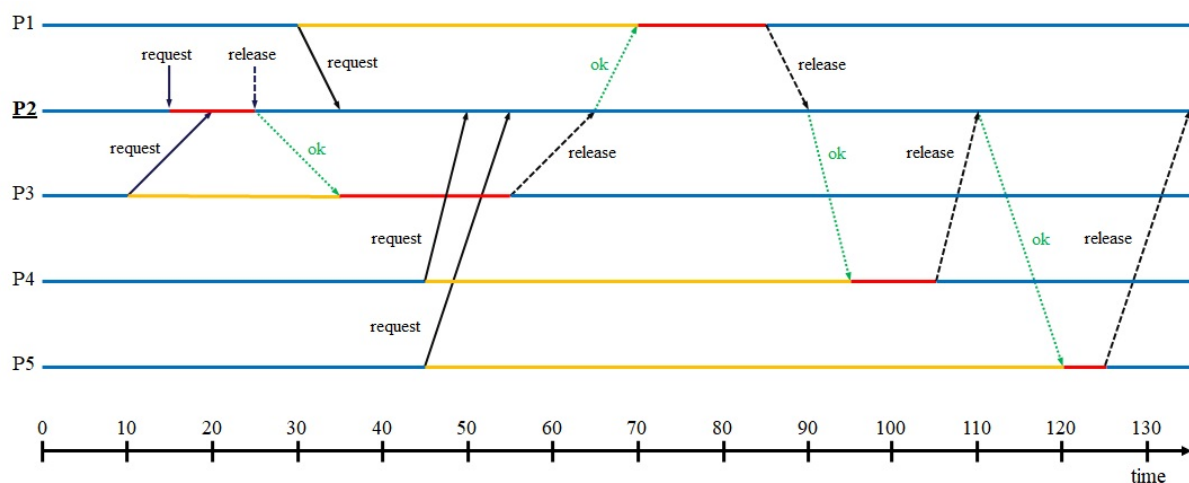


c) Sigui $C(e)$ el rellotge lògic de Lamport d'un event e , algú ens assegura que si $C(a) < C(b)$, llavors segur que b no ha passat abans que a . Justifica si aquesta afirmació és correcta.

Solution: It is correct. $b \rightarrow a$ would imply that $C(b) < C(a)$. Given that $C(a) < C(b)$, we know that $a \rightarrow b$ or $a \parallel b$

5. (1.25 pts) Tenim un conjunt de cinc processos (P1–P5) que coordinen els seus accessos a una regió crítica mitjançant l'algorisme centralitzat on el procés P2 ha estat escollit coordinador. La següent taula detalla per cada procés en quin instant de temps demana accedir a la regió crítica (columna 1), quant temps treballa dins de la regió crítica abans d'alliberar-la (columna 2), i la latència de comunicació amb el coordinador (columna 3).

	temps petició accés	temps dins regió crítica	latència amb coordinador
P1	30	15	5
P2	15	10	0
P3	10	20	10
P4	45	10	5
P5	45	5	10



- a) Ajuda't del cronograma per calcular en quin instant de temps accedirà cada procés a la regió crítica.

Solution:

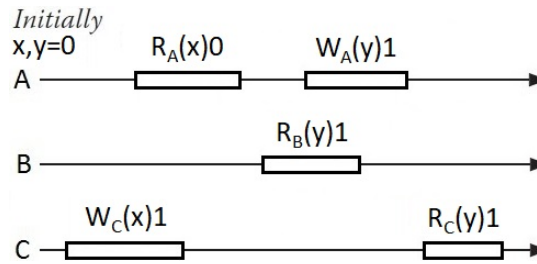
P1 accesses the critical section at $t = 70$.
 P2 accesses the critical section at $t = 15$.
 P3 accesses the critical section at $t = 35$.
 P4 accesses the critical section at $t = 95$.
 P5 accesses the critical section at $t = 120$.

- b) Explica com aquest algorisme satisfà la propietat de *liveness* (i.e. una petició per accedir a la regió crítica serà concedida eventualment).

Solution: If the critical section is free, the coordinator grants the request right-away. Otherwise, it queues the request. As queued requests are granted in their order of arrival when the critical section becomes free, all of them will eventually succeed to access.

Nom i Cognoms:

6. (1.25 pts) Donats 3 clients A, B i C que executen les següents operacions en un *data store* replicat, justifica si aquesta execució satisfà el model de consistència de *linearizability*.



Solution: The execution is linearizable because there exists an interleaving of all the operations which is legal (meets the specification of a single correct copy of the objects), is consistent with the real times at which the operations occurred when they do not overlap, and makes operations visible to everyone before they return. E.g., $R_A(x)0 \prec W_C(x)1 \prec W_A(y)1 \prec R_B(y)1 \prec R_C(y)1$

Si el *data store* té 8 rèpliques i usa un protocol d'escriptura replicada basat en quòrum (*quorum-based replicated-write*) en què el temps de *round-trip* (RTT) dels clients per accedir a cada rèplica és:

	RTT des de A	RTT des de C		RTT des de A	RTT des de C
R_1	7 ms	2 ms	R_5	3 ms	6 ms
R_2	2 ms	7 ms	R_6	4 ms	3 ms
R_3	5 ms	4 ms	R_7	2 ms	7 ms
R_4	1 ms	5 ms	R_8	6 ms	3 ms

- a) Raona com fixar N_R i N_W per evitar tant conflictes RD-WR com WR-WR i escull valors concrets que serveixin per optimitzar les operacions d'escriptura (ahora que s'eviten els conflictes).

Solution: $N_W > 4$ to avoid WR-WR conflicts ; $N_R + N_W > 8$ to avoid RD-WR conflicts
Write operations can be optimized minimizing the size of N_W . The minimum value of N_W that avoids WR-WR conflicts is 5. According to this, N_R must be 4.

- b) Enumera quines rèpliques s'han d'incloure en els quòrums N_W i N_R de les operacions $W_C(x)1$ i $R_A(x)0$ si es pretén minimitzar el seu temps de *round-trip*, i indica quin seria aquest temps, tenint en compte que les rèpliques del quòrum es poden contactar en paral·lel.

Solution: $R_A(x)0$: RTT would be 3 ms by picking these 4 replicas R_2, R_4, R_5, R_7 in N_R .
 $W_C(x)1$: RTT would be 5 ms by picking these 5 replicas R_1, R_3, R_4, R_6, R_8 in N_W .

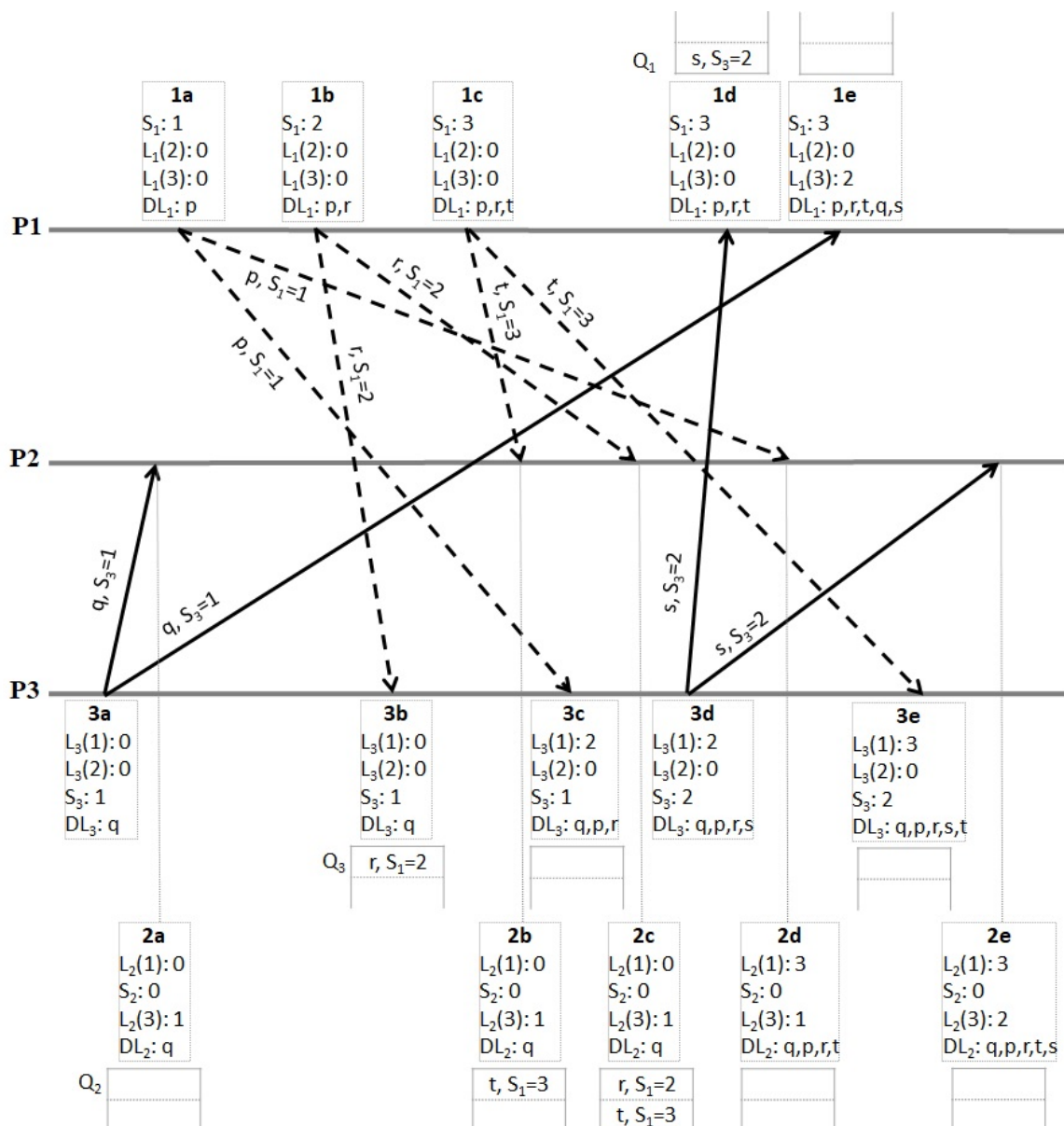
- c) Calcula quantes fallades de crash pot suportar aquest *data store* en les operacions de lectura i escriptura donada la definició concreta de N_R i N_W que has escollit.

Solution: Can tolerate up to 3 crash failures for write operations: $K + N_W \leq 8$; $K \leq 3$.
Can tolerate up to 4 crash failures for read operations: $K + N_R \leq 8$; $K \leq 4$.

- d) Calcula quantes fallades Byzantines pot suportar aquest *data store* en les operacions de lectura i escriptura i com caldria adaptar el càlcul de N_R i N_W en funció d'aquest nombre de fallades.

Solution: Can tolerate up to 2 Byzantine failures: $3K + 1 \leq 8$; $K \leq (8 - 1)/3$; $K \leq 2$.
 $N_W > (N + K)/2 = (8 + 2)/2 = 5$ to avoid WR-WR conflicts
 $N_R + N_W > N + K = 8 + 2 = 10$ to avoid RD-WR conflicts

7. (1.25 pts) Tenim el següent grup de processos que es comuniquen mitjançant multicast fiable amb ordenació FIFO (*FIFO-ordered reliable multicast*). Indica per cada procés i : el número de seqüència del darrer missatge que ha lliurat procedent de qualsevol altre procés j ($L_i(j)$), els missatges guardats a la *hold-back queue* pendents de ser lliurats (Q_i), la llista de missatges lliurats fins el moment (DL_i), i el número de seqüència del darrer missatge multicast enviat (S_i). Tenint en compte que el multicast fiable usa la implementació bàsica, omple també la taula indicant per cada procés en quins instants de temps envia les confirmacions ACK i/o NACK corresponents a cada missatge multicast (p.ex. si P1 envia un ACK pel missatge p en l'instant $1a$, posaríem 'ACK: 1a' a la cel·la (P1,p)). NOTA: i) si un procés ja ha enviat un NACK referent a un missatge concret amb anterioritat, no ho tornarà a fer, ii) un procés no enviarà cap ACK pels missatges multicast que ell mateix ha enviat.



	p	q	r	s	t
P1		NACK: 1d; ACK: 1e		ACK: 1e	
P2	NACK: 2b; ACK: 2d	ACK: 2a	NACK: 2b; ACK: 2d	ACK: 2e	ACK: 2d
P3	NACK: 3b; ACK: 3c		ACK: 3c		ACK: 3e

Nom i Cognoms:

8. (READINGS) Contesta les següents preguntes referents als articles llegits a l'assignatura.

- i) Explica quines dues característiques d'Erlang poden ajudar a afrontar l'assumpció errònia 'The Network Is Reliable' segons es descriu a l'article *Hebert13*.

Solution: On one side, Erlang has specific functions to monitor nodes and detect them getting disconnected (or becoming unresponsive). On the other side, Erlang uses asynchronous communication and forces developers to send a reply back when things work well, which pushes for all message-passing activities to intuitively handle failure.

- ii) Defineix el concepte de *synchronization* i indica quin requeriment de sincronització tenen els sistemes que usen Precision Time Protocol (PTP) segons l'article *Neville-Neil16*.

Solution: Synchronization is how close two different clocks are to each other at any particular instant. PTP is used in systems where submicrosecond synchronization is necessary.

- iii) Justifica per què el model *eventual consistency* proporciona millor rendiment i disponibilitat a les operacions de lectura que el model *strong consistency* segons es descriu a l'article *Terry13*.

Solution: Strong consistency generally requires reading from a designated primary site or from a majority of replicas, whereas eventual consistency allows clients to read from any replica. With more choices of replicas, clients are more able to choose a nearby one with lower latency, and also more likely to find one that is reachable.